

1. Objetivo

Lanzar un *kernel* (función ejecutada en la tarjeta gráfica) de un solo bloque con múltiples hilos o *threads*.

2. Ejercicio

Calcular la suma de dos vectores de tamaño *N*. El programa debe cumplir los siguientes requisitos:

- El tamaño *N* debe estar definido en tiempo de ejecución por el usuario.
- Uno de los vectores se debe generar en el *host* con *N* valores aleatorios de tipo `int` comprendidos entre 0 y 9.
- El segundo vector se debe generar en el *device* y será el inverso del primer vector, es decir, los mismos valores pero en orden inverso.
- La suma será realizada en el *device* aprovechando todos los hilos o *threads* lanzados sin sobrepasar el máximo permitido.
- Por pantalla se debe mostrar:
  - Las principales propiedades del dispositivo seleccionado (nombre, capacidad de cómputo, número de multiprocesadores, número de núcleos y tamaño de la memoria global en MiB).
  - Los dos vectores iniciales y el vector con el resultado final.

3. Elementos CUDA

Para realizar esta práctica se puede utilizar cualquiera de las funciones estudiadas anteriormente junto con los siguientes elementos:

- **`__global__`**

  Etiqueta o especificador que se coloca junto a la declaración de la función *kernel* para indicar al compilador que dicha función se va a ejecutar en el *device*.

- **`<<<num_bloques,num_hilos>>>`**

  Sintaxis de llamada a la función *kernel* para indicar el número de copias que se desea lanzar.

- **`threadIdx.x`**

  Variable de tipo `int` que se genera automáticamente dentro de la función *kernel* y que permite identificar cada una de las copias que se han lanzado.

3. Solución propuesta

```
/*
* ARQUITECTURA DE COMPUTADORES
* 2º Grado en Ingenieria Informatica
* Introduccion a la programacion en CUDA
*
* Basico 2: "Funcion kernel"
* >> Suma de dos vectores de tamaño n
*
*/
////////////////////////////////////////////////////////////////////////
// includes
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <cuda_runtime.h>

////////////////////////////////////////////////////////////////////////
// declaracion de funciones
// HOST: funcion llamada desde el host y ejecutada en el host
__host__ int propiedades_Device();
// Funcion que imprime las propiedades del device

// GLOBAL: funcion llamada desde el host y ejecutada en el device (kernel)
__global__ void suma(int *vector_1, int *vector_2, int *vector_suma, int n)
// Funcion que suma dos vectores de n elementos
{
	// KERNEL de 1 BLOQUE
	// identificador de hilo
	int myID = threadIdx.x;
	// obtenemos el vector 2 invirtiendo el vector 1
	vector_2[myID] = vector_1[n - 1 - myID];
	// sumamos los dos vectores y escribimos el resultado
	vector_suma[myID] = vector_1[myID] + vector_2[myID];
}
////////////////////////////////////////////////////////////////////////
// MAIN: rutina principal ejecutada en el host
int main(int argc, char** argv)
{
	// buscando dispositivos
	int maxThreads;
	int deviceCount;
	cudaGetDeviceCount(&deviceCount);
	if (deviceCount == 0)
	{
		printf("!!!!!No se han encontrado dispositivos CUDA!!!!!\n");
		printf("<pulsa [INTRO] para finalizar>");
		getchar();
		return 1;
	}
	else
	{
		maxThreads = propiedades_Device();
	}
	// declaraciones
	int *hst_vector1, *hst_vector2, *hst_resultado;
```

```
	int *dev_vector1, *dev_vector2, *dev_resultado;
	int n;

	// pedimos por teclado el numero de elementos
	printf("  Introduce el numero de elementos: ");
	scanf("%d", &n);
	// capturamos el [INTRO] de la funcion scanf()
	getchar();
	while (n > maxThreads)
	{
		printf("> ERROR: numero maximo de hilos superado! [%d hilos]\n", maxThreads);
		printf("  Introduce el numero de elementos: ");
		scanf("%d", &n);
		// capturamos el [INTRO] de la funcion scanf()
		getchar();
	}

	// reserva en el host
	hst_vector1   = (int*)malloc(n * sizeof(int));
	hst_vector2   = (int*)malloc(n * sizeof(int));
	hst_resultado = (int*)malloc(n * sizeof(int));

	// reserva en el device
	cudaMalloc((void**)&dev_vector1, n * sizeof(int));
	cudaMalloc((void**)&dev_vector2, n * sizeof(int));
	cudaMalloc((void**)&dev_resultado, n * sizeof(int));

	// inicializacion de vectores
	for (int i = 0; i < n; i++)
	{
		hst_vector1[i] = rand() % 10;
	}

	// copia de datos hacia el device
	cudaMemcpy(dev_vector1, hst_vector1, n * sizeof(int), cudaMemcpyHostToDevice);

	// LANZAMIENTO DEL KERNEL
	printf("> Vector de %d elementos\n", n);
	printf("> Lanzamiento con %d bloque de %d hilos\n", 1, n);
	suma << < 1, n >> >(dev_vector1, dev_vector2, dev_resultado, n);
	// recogida de datos desde el device
	cudaMemcpy(hst_vector2, dev_vector2, n * sizeof(int), cudaMemcpyDeviceToHost);
	cudaMemcpy(hst_resultado, dev_resultado, n * sizeof(int), cudaMemcpyDeviceToHost);

	// impresion de resultados
	printf("VECTOR 1:\n");
	for (int i = 0; i < n; i++)
	{
		printf("%2d ", hst_vector1[i]);
	}
	printf("\n");
	printf("VECTOR 2:\n");
	for (int i = 0; i < n; i++)
	{
		printf("%2d ", hst_vector2[i]);
	}
	printf("\n");
	printf("SUMA:\n");
	for (int i = 0; i < n; i++)
	{
		printf("%2d ", hst_resultado[i]);
	}
	printf("\n");
	// salida
	time_t fecha;
	time(&fecha);
	printf("************************************************\n");
	printf("Programa ejecutado el: %s\n", ctime(&fecha));
	printf("<pulsa [INTRO] para finalizar>");
	getchar();
	return 0;
}
////////////////////////////////////////////////////////////////////////
// HOST: funcion llamada desde el host y ejecutada en el host
__host__ int propiedades_Device()
{
	int deviceID;
	cudaGetDevice(&deviceID);
	int runtimeVersion;
	cudaRuntimeGetVersion(&runtimeVersion);
	cudaDeviceProp deviceProp;
```

```
        cudaGetDeviceProperties(&deviceProp, deviceID);
        // calculo del numero de cores (SP)
        int cudaCores = 0;
        int SM = deviceProp.multiProcessorCount;
        int major = deviceProp.major;
        int minor = deviceProp.minor;
        const char *archName;
        switch (major)
        {
        case 1:
                //TESLA
                archName = "TESLA";
                cudaCores = 8;
                break;
        case 2:
                //FERMI
                archName = "FERMI";
                if (minor == 0)
                        cudaCores = 32;
                else
                        cudaCores = 48;
                break;
        case 3:
                //KEPLER
                archName = "KEPLER";
                cudaCores = 192;
                break;
        case 5:
                //MAXWELL
                archName = "MAXWELL";
                cudaCores = 128;
                break;
        case 6:
                //PASCAL
                archName = "PASCAL";
                cudaCores = 64;
                break;
        case 7:
                //VOLTA (7.0) TURING (7.5)
                cudaCores = 64;
                if (minor == 0)
                        archName = "VOLTA";
                else
                        archName = "TURING";
                break;
        case 8:
                //AMPERE
                archName = "AMPERE";
                cudaCores = 64;
                break;
        default:
                //ARQUITECTURA DESCONOCIDA
                archName = "DESCONOCIDA";
                cudaCores = 0;
        }
        // presentacion de propiedades
        printf("************************************************\n");
        printf("DEVICE %d: %s\n", deviceID, deviceProp.name);
        printf("************************************************\n");
        printf("> CUDA Toolkit                      \t: %d.%d\n", runtimeVersion / 1000,
(runtimeVersion % 1000) / 10);
        printf("> Arquitectura CUDA                 \t: %s\n", archName);
        printf("> Capacidad de Computo              \t: %d.%d\n", major, minor);
        printf("> No. de MultiProcesadores          \t: %d\n", SM);
        printf("> No. de CUDA Cores (%dx%d)         \t: %d\n", cudaCores, SM, cudaCores*SM);
        printf("> No. maximo de Hilos (por bloque)\t: %d\n", deviceProp.maxThreadsPerBlock);
        printf("> Memoria Global (total)            \t: %zu MiB\n", deviceProp.totalGlobalMem /
(1024 * 1024));
        printf("************************************************\n");

        // salida
        return deviceProp.maxThreadsPerBlock;
}
//////////////////////////////////////////////////////////////////////////
```